Name: Khushi Chhaproo | Roll No: 07 | Div: D20C

Blockchain Experiment No. 6

Aim:

Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE (Voting Ballot).

Theory:

Relevance of Required Statements in Solidity Programs:

In Solidity, the **required** statement acts as a guard condition within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside **require** is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt the blockchain data.

For example, in a Voting Smart Contract, **require** can be used to check:

- Whether the person calling the function has the right to vote (`require(voters[msg.sender].weight > 0, "Has no right to vote");`).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the chairperson before granting voting rights.

Thus, **require** statements enforce security, correctness, and reliability in smart contracts. They also allow developers to attach error messages, making debugging and contract interaction easier for users.

Keywords: mapping, storage, and memory

1. mapping:

A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is `mapping(keyType => valueType)`. For example:

```
mapping(address => Voter) public voters;
```

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful for contracts like Ballot, where you need to associate voters with their data (whether they voted, which proposal they chose, etc.). Unlike arrays, mappings do not have a length property and cannot be iterated over directly, making them gas efficient for lookups but limited for enumeration.

2. storage:

In Solidity, **storage** refers to the permanent memory of the contract, stored on the Ethereum blockchain. Variables declared at the contract level are stored in storage by default. Data stored in storage is persistent across transactions, which means once written, it remains available unless explicitly modified. However,

because writing to blockchain storage consumes gas, it is more expensive. For example, a voter's information saved in the voters mapping remains available throughout the contract's lifecycle.

3. memory:

In contrast, **memory** is temporary storage, used only for the lifetime of a function call. When the function execution ends, the data stored in memory is discarded. Memory is mainly used for temporary variables, function arguments, or computations that don't need to be permanently stored on the blockchain. It is cheaper than storage in terms of gas cost. For instance, when handling proposal names or temporary string manipulations, memory is often used.

Thus, a smart contract developer must balance between **storage** and **memory** to ensure efficiency and costeffectiveness.

Why bytes32 instead of string?

In earlier implementations of the Ballot contract, **bytes32** was used for proposal names instead of **string**. The reason lies in efficiency and gas optimization.

bytes32 is a fixed-size type, meaning it always stores exactly 32 bytes of data. This makes storage characters, which is not very flexible for user-friendly names simple, comparison operations faster, and gas costs

string is a dynamically sized type, meaning it can store text of variable length. While it is easier for the Ethereum Virtual Machine (EVM). This increases gas usage and may slow down comparisons or d velopers and users (since names and be written manipulations.

To make the system more user-friendly, modern implementations of the Ballot contract often convert from **bytes32** to **string**. Tools like the Web3 Type Converter help developers easily switch between these two types for deployment and testing.

In summary, **bytes32** is used when performance and gas efficiency are priorities, while **string** is preferred for readability and ease of use.

Code:

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.20;

contract VotingBallot {

    struct Candidate {
   string name;
   uint256 voteCount;
   }

    Candidate[] public candidates;
    mapping(address => bool) public hasVoted;

   address public owner;

    event CandidateAdded(
   uint256 indexed candidateId,
   string name  );

    event VoteCast(  address
   indexed voter,  uint256
   indexed candidateId  );
```

```
 modifier onlyOwner() {
require(
     msg.sender == owner,
     "Only owner can perform this action" );
 _;
 }

 constructor() {
owner = msg.sender;
}

 function addCandidate(
string memory _name  )
public onlyOwner {
require(
     bytes(_name).length > 0,
     "Candidate name cannot be empty"
 );


    candidates.push(
    Candidate({
    name: _name,
    voteCount: 0
     })
     );

 emit CandidateAdded(
     candidates.length - 1,
     _name
 );
 }

 function vote(
uint256 _candidateId
 ) public {
require(
     !hasVoted[msg.sender],
     "You have already voted"
 );

     require(  _candidateId <
    candidates.length,
```



```
     "Invalid candidate"
     );  hasVoted[msg.sender] = true;

    candidates[_candidateId].voteCount++;

 emit VoteCast(
   msg.sender,
    _candidateId
 );
 }

 function getCandidateCount()
 public  view
returns (uint256)
```

```
    {
    return candidates.length;
   }

    function getCandidate(
   uint256 _candidateId
    )  public  view
   returns (  string
   memory,  uint256
    )  {
   require(
        _candidateId < candidates.length,
        "Invalid candidate"
    );

    Candidate memory candidate =
   candidates[_candidateId];  return (
        candidate.name,
       candidate.voteCount
    );
    }

    function getWinner()
   public  view  returns
   (
        string memory winnerName,
       uint256 winnerVotes
    )  {
   require(
        candidates.length > 0,
       "No candidates available"
    );
        uint256 winningIndex = 0;

        for (  uint256 i = 1;
       i < candidates.length;
        i++  ){  if (
       candidates[i].voteCount >
       candidates[winningIndex].voteCount
        ){  winningIndex =
           i;
        }
       }

        winnerName =
       candidates[winningIndex].name;

        winnerVotes =
        candidates[winningIndex].voteCount; }
}
```

Output:

Contract Compiled

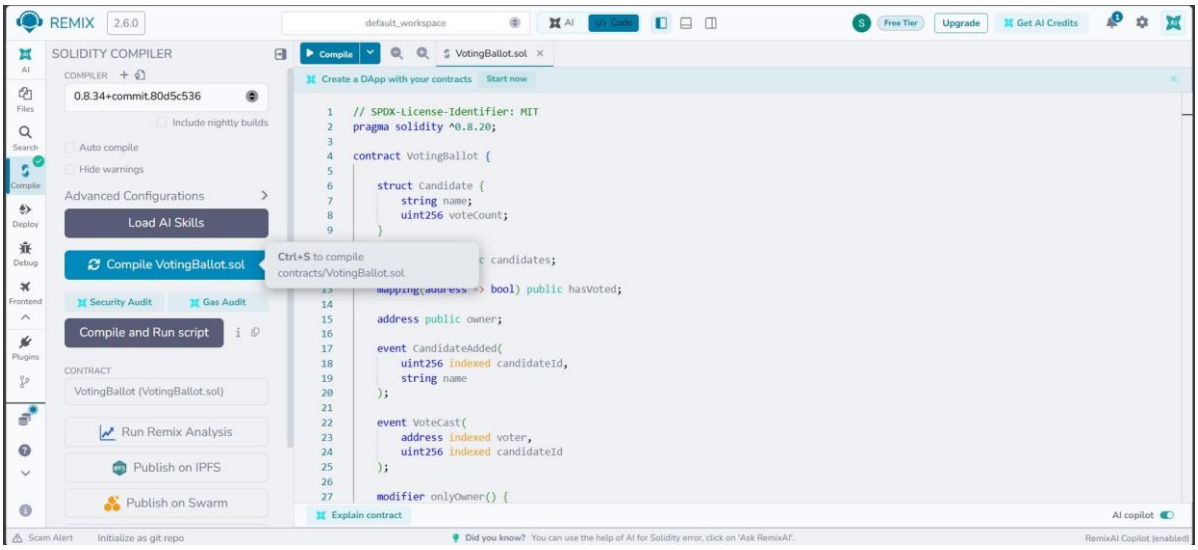


Figure 1 — Contract Compiled — Remix IDE screen.

Contract Deployed

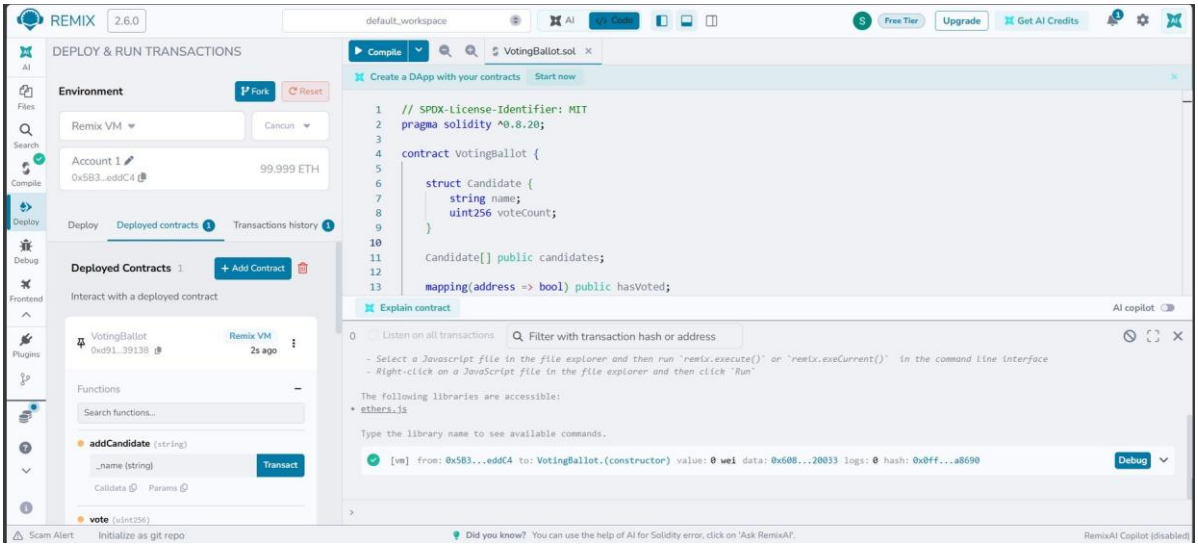


Figure 2 — Contract Deployed — Remix IDE screen. **Owner of contract**

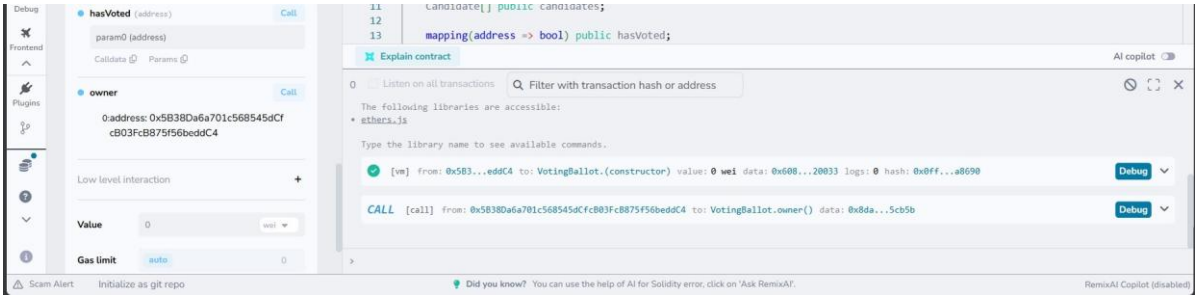


Figure 3 — Owner of contract — Remix IDE screen. **Added**

Alice as candidate

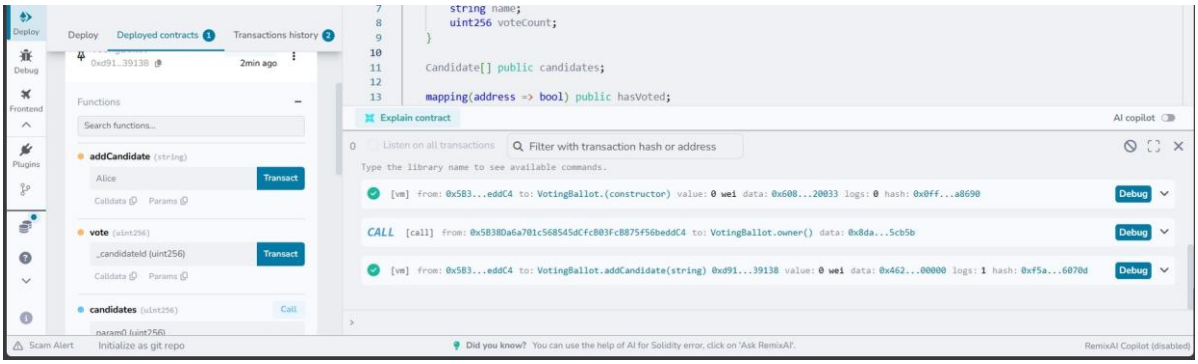


Figure 4 — Added Alice as candidate — Remix IDE screen.

Added 2nd Candidate Bob

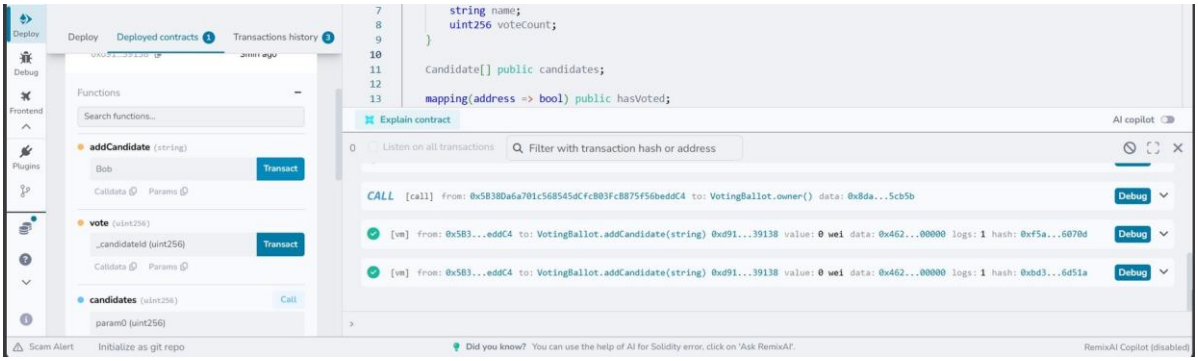


Figure 5 — Added 2nd Candidate Bob — Remix IDE screen. **Added**

3rd candidate Charlie

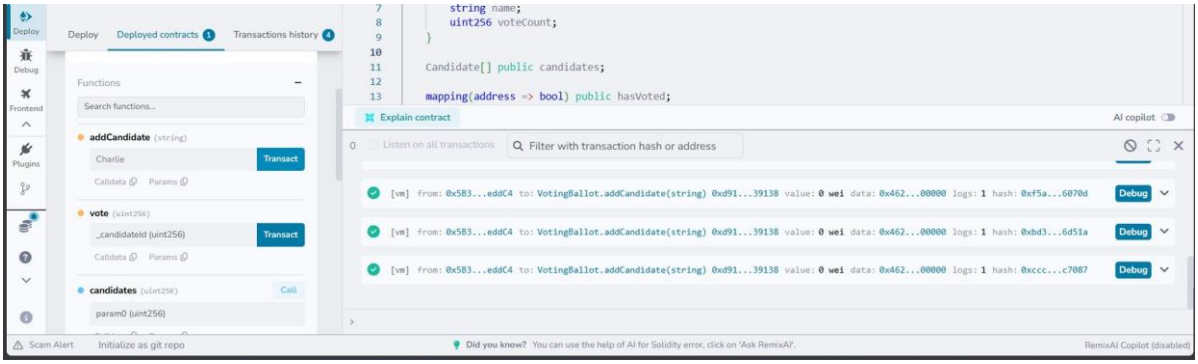


Figure 6 — Added 3rd candidate Charlie — Remix IDE screen.

Calling getCandidateCount (output is 3)

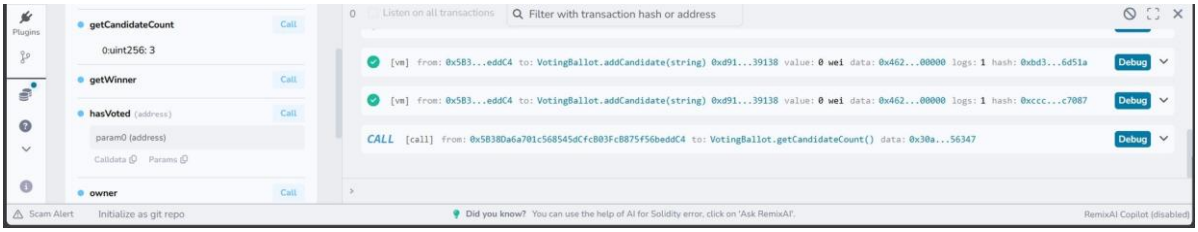


Figure 7 — Calling getCandidateCount (output is 3) — Remix IDE screen.

Calling getCandidate function on Alice (Alice has 0 votes)

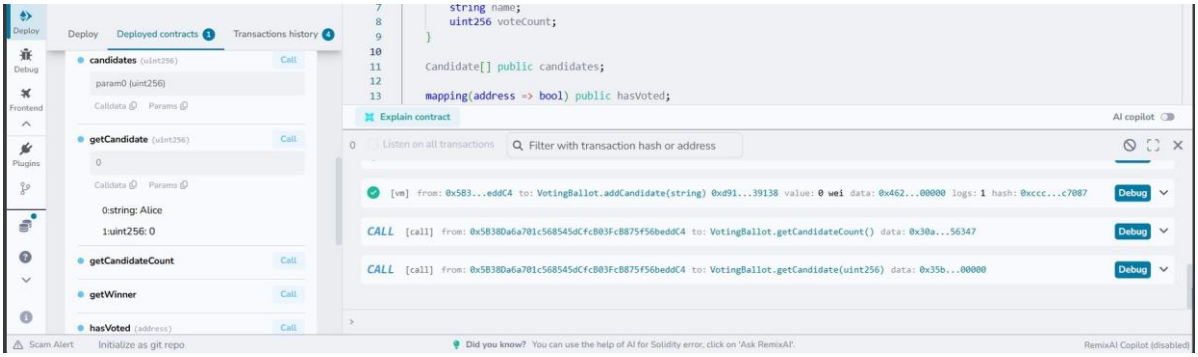


Figure 8 — Calling getCandidate function on Alice (Alice has 0 votes) — Remix IDE screen.

Voting for Alice using Account 2

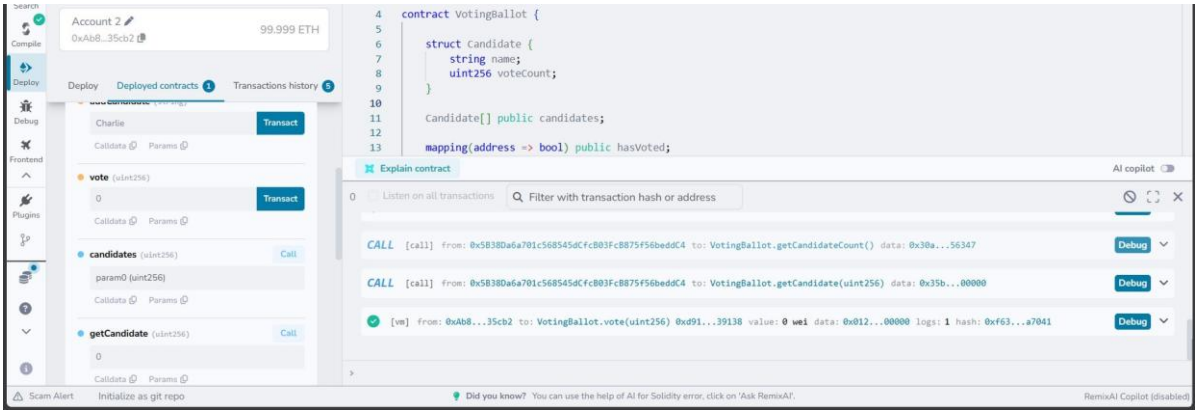


Figure 9 — Voting for Alice using Account 2 — Remix IDE screen.



Figure 10 — Voting for Alice using Account 2 — Remix IDE screen.

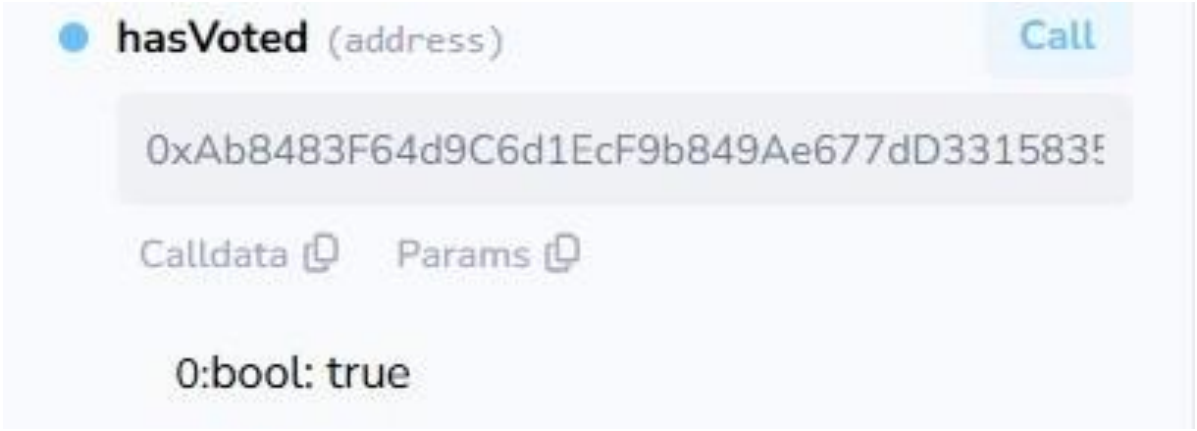


Figure 11 — Voting for Alice using Account 2 — Remix IDE screen.

Verify that the vote is given to Alice

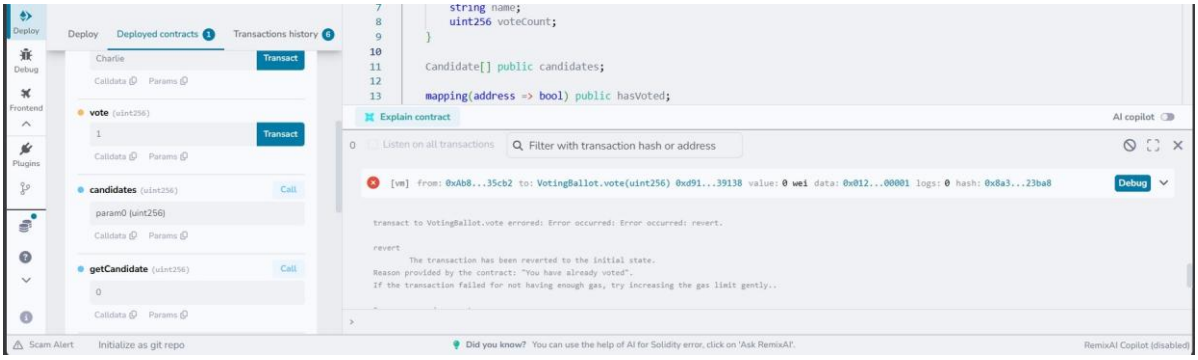


Figure 12 — Verify that the vote is given to Alice — Remix IDE screen.

Verify Account 2 has voted



Figure 13 — Verify Account 2 has voted — Remix IDE screen.

Preventing double voting

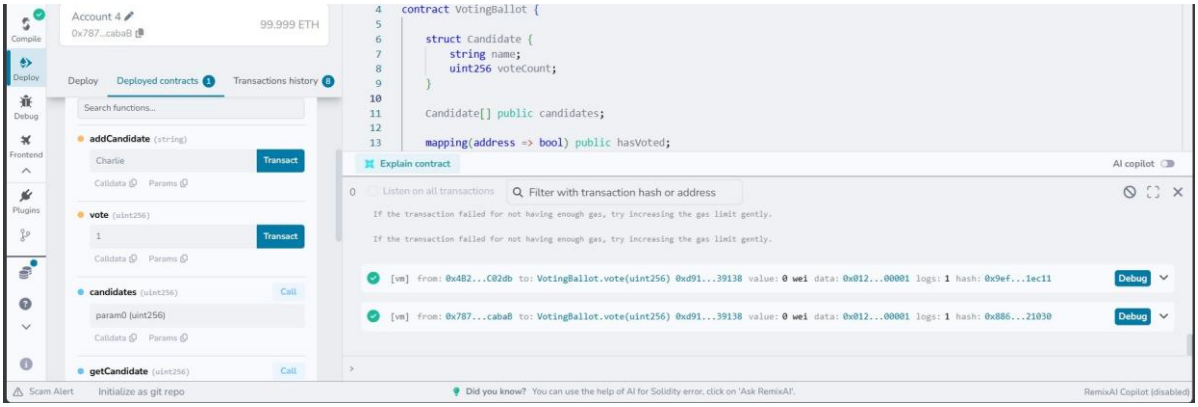


Figure 14 — Preventing double voting — Remix IDE screen. **Using**

3rd and 4th account to vote for Bob(1)

Figure 15 — Using 3rd and 4th account to vote for Bob(1) — Remix IDE screen.

Notes from the execution:



GDouble voting prevented successfully.

GFinding the winner using getWinner returned the candidate with the highest vote count after the votes cast by Accounts 2, 3, and 4.

Conclusion:

In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of **require** statements, mapping, and data location specifiers like **storage** and **memory** were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts. The difference between using **bytes32** and **string** for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchain.